SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13

CO3-AT(3)

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Duration: 1 Hour

Total Marks:25

CASE STUDY

Code of Conduct:

I Om Sai Reddy T / 192372252 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Consider a case study in parsing nested structured expressions using a Context-Free Grammar (CFG). Given the grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S,$

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. **Designing a Grammar for Arithmetic Expressions**
   A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as a+b*c.

Case Study Question:
Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

---

2. **Mini Programming Language Parser Development**
   A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:
Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

---

3. **Grammar Optimization for SQL Query Validation**
   A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:
Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

---

4. **Natural Language Processing Chatbot**
   A chatbot system processes simple English sentences such as:
   Subject + Verb + Object
   Example: "Students learn TOC."
   The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:
Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

---

5. **Parser Design for HTML-like Markup Language**
   A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:
Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. Designing a Grammar for Arithmetic Expressions

Case Study

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as a+b*c.

Answer

Ambiguous Grammar

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

This grammar is ambiguous because the expression a+b*c can have multiple parse trees.

Optimized CFG

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T/F \mid F$$
$$F \rightarrow (E) \mid id$$

Optimization Performed

- Separate non-terminals for precedence levels.
- T handles multiplication/division.
- E handles addition/subtraction.
- Removes ambiguity.
- Supports efficient top-down or bottom-up parsing.

---

2. Mini Programming Language Parser Development

Case Study

A startup is designing a small scripting language for automating IoT devices. The parser frequently reports syntax conflicts during compilation.

Answer

Initial Grammar with Left Recursion

$$Stmt \rightarrow Stmt\ stmt \mid stmt$$

Optimized CFG

$$Stmt \rightarrow stmt\ Stmt$$
$$Stmt \rightarrow stmt\ Stmt \mid \epsilon$$

CFG for Language Constructs

$$Loop \rightarrow while(condition)\{Stmt\}$$
$$Cond \rightarrow if(condition)\{Stmt\}$$

Optimization Performed

- Eliminated left recursion.
- Reduced shift-reduce conflicts.
- Improved predictive parsing.

---

3. Grammar Optimization for SQL Query Validation

Case Study

A database company is developing a query validation tool for simplified SQL commands.

Answer

Initial Grammar

$$Q \rightarrow SELECT\ A\ FROM\ T\ WHERE\ C$$

Optimized CFG

$$Q \rightarrow SELECT\ Fields\ FROM\ Table\ OptWhere$$
$$OptWhere \rightarrow WHERE\ Condition \mid \epsilon$$

Optimization Performed

- Introduced optional productions.
- Reduced redundancy.
- Simplified parser design.
- Improved syntax validation speed.

Example Accepted Query

SELECT name FROM student WHERE mark > 50

---

4. Natural Language Processing Chatbot

Case Study

A chatbot processes simple English sentences like "Students learn TOC."

Answer

CFG Design

$$S \rightarrow NP\ VP$$
$$NP \rightarrow Noun$$
$$VP \rightarrow Verb\ NP$$
$$Noun \rightarrow Students \mid TOC$$
$$Verb \rightarrow learn$$

Optimization Performed

- Restricted sentence order.
- Invalid patterns are rejected.
- Improved grammatical correctness.

Valid Sentence

Students learn TOC

Invalid Sentence

Learn Students TOC

---

5. Parser Design for HTML-like Markup Language

Case Study

A web editor company is building a parser for a simplified markup language with nested tags.

Answer

CFG for Nested Tags

$$S \rightarrow < tag > S < /tag > \mid text \mid \epsilon$$

Example

```
<tag>
  <tag>text</tag>
</tag>
```

Optimization Performed

- Recursive production supports nesting.
- Ensures proper opening and closing tags.
- Detects invalid nesting structures.

Invalid Example

<tag>text</tag>

Benefit

Q) Designing a Grammar for Arthemetic Expressions

A) CFG [Context Free Grammar]

=*=

$E \rightarrow E+T \mid E-T \mid T$

$T \rightarrow T*F \mid T \mid F \mid F$

$F \rightarrow (E) \mid id$

Where :-

=*=*=

E = Expression

T = Term

F = Factor

id = number / identifier.

Explanation :-

=*=*=*=

This grammar gives higher precedence to * and / than + and - .

Example :-

=*=*=

a+b*c

is parsed as :-

=*=*=+=

a + (b*c)

and not :-

=*=+=

(a+b)*c

It also provides left associativity :-

=*=*=*=*=+=*=+=

a - b - c

is interpreted as :-

=+=*=*=*=

(a-b)-c

For a top-down Parser, remove left recursion :-

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid E$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid FT' \mid E$

$F \rightarrow (E) \mid id.$

Conclusion :-

The grammar removes ambiguity by separating the expression, term and factor levels. These ensures correct precedence and efficient parsing.

(2)

A) Mini Programming language parser :-

Assume the language supports :-

*) Variable declaration.

*) Assignment

*) If

*) while.

Simple CFG :-

Program → Statement list.

Statement list → Statement Statement list | ϵ.

statement → Declaration

| Assignment

| if statement

| while statement

Declaration → Type id;

Type → int | float | bool.

Assignment → id = Expression;

if statement → if (Expression) { statement list }

While statement → while (Expression) { statement list }

Expression → id | Number.

Ex:-

```
int x;
x=10
if (x) {
    x=20;
}
While (x) {
    x=30;
}
```

The grammar recognizes declarations, assignments, conditions and loops.

Optimization:-

=+=+=+=

The original grammar may contain left recursion such as:-

$$E \rightarrow E+T \mid T.$$

We Convert it to :-

=* =* =* =* =

$$E \rightarrow TE'$$

$$E' \rightarrow TE' \mid \varepsilon.$$

Conclusion :-

=* =* =*

The optimized grammar reduces parsing conflicts, avoids left recursion, and makes syntax analysis faster and simpler.

---

③ Grammar optimization for SQL Query Validation

Select name, age from Students where age > 18;

A)

CF G :-

=* =

Query :- Select columnlist from table where clause;

Columnlist → 'id columnlist'

Columnlist → id Coloumn list' | E.

Table → id.

Where Clause → WHERE Condition | E.

Condition → id Operator Value.

Operator → = | > | < | >= | <= | <>

Value → id | number.

Valid Query :-

Select name, age from students where age > 18;

It follows :-

Select columnlist from table whereclause.

Query without where :-

Select name from students ;

whereclause → E.

Invalid Query :-

Select from students;

Optimization :-

Instead of using unnecessary productions such as :-

Query → Select Query

Select Query → ~~Query~~ (1)

Query (1) → select

Query → Select Column list from table where clause;

Conclusion :-

The optimized SQL grammar removes redundant productions and ambiguity.

---

(4) Natural Language processing Chatbot :-

A) Subject + Verb + Object.

Ex :-

Students learn TOC.

CFG :-

=*=

S → plural | sentence | singular Sentence.

plural Sentence → plural subject plural verb Object.

Singular Sentence → Singular subject singular verb Objd.

plural Subject → students | Teachers.

Singular Subject → student | Teacher.

plural verb → learn | study.

Singular Verb → learns | studies.

Object → Toc | Mathematics | Programming.

Valid Sentence :-

=*=*=*=*=

Students learn Toc

Because :-

=*=*=

plural Subject → Students

Plural Verb → learn.

Object → Toc

Therefore :-

=*=*=

Students + lean + Toc

is valid.

Conclusion :-

=*=*=*=

The grammar improves validation by separating singular and plural subjects and verbs.

Design for HTML - like Markup language

<book>

</book>

Q:-

Document → Element

Element → <books Content </books>

| <title> Content </title>

| <author> Content </author>

Content → Element content.

| text content.

| E

Valid Example :-

<book>

<title>

Toc

</title>

</book>

<book> → content → <title> → content →

</title> → </book>.

Another

<book>
<author>
<title>
TOC
</title>
</author>
</book>

Invalid Example

<book>
<title>
</book>
</title>

This is rejected because <title> must be closed before </book>.

The recursive rule:-

Content → Element Content.

Correct:-

<book>
<title>
</title>
</book>

Optimization:-

Content → Element Content | text Content | E.

Conclusion:-

The recursive CFG correctly validates Opening and closing tags and supports nested elements.

SIMATS ENGINEERING
ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (25 Marks)	Level 3 – Proficient (20-24 Marks)	Level 2 – Developing (10–15 Marks)	Level 1 – Beginning (5 -10 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

91/100